NixNan: Detecting Floating-Point Exceptions on GPUs

Architecture, Features, and Four Case Studies

SC26 Tutorial

Why Floating-Point Exceptions Matter

- FP exceptions (NaN, Inf) are **almost always bugs**, not intended results.
- They are easy to generate, especially in dot products / reductions, and especially in **low-precision ML formats**:

$$(1.9 \times 10^{19})^2 \to \texttt{INF} \text{ in FP32}, \qquad 16^2 = \texttt{INF} \text{ in E4M3 (a popular ML format)}$$

- A classic, deceptively simple bug:

```
#define MAX(x,y) ( (x >= y) ? x : y )
```

  With $x = \texttt{NaN}, y = 42$: result is 42 (looks clean). Swap the arguments: result is NaN. **MAX must be commutative — it isn't.**

- NaNs can silently skew **control flow**, not just numeric output.

The Silent-Killer Problem, and Why GPUs Make It Worse

- Exceptions often **vanish harmlessly** on the way to program output (masked by a later operation, a reduction, a clamp) — but it is very hard to know, after the fact, **which control-flow decisions they influenced** while they were alive.
- Results are only easy to observe at the very *end* of a kernel or program. If an exception dies before reaching there, the programmer never sees it.
- NVIDIA GPUs generate **no hardware traps** on FP exceptions — unlike a CPU's SIGFPE.
- So exception detection must happen **in software**, by decoding the bit pattern of every computed value:

$$\underbrace{S}_{\text{sign}}\ \underbrace{E\cdots E}_{\text{exponent}}\ \underbrace{F\cdots F}_{\text{mantissa}}$$

- All-ones exponent + nonzero mantissa = NaN; all-ones exponent + zero mantissa = $\pm\infty$; all-zeros exponent + nonzero mantissa = subnormal.
- This has to happen for **every FP instruction, every thread, every launch** — at native GPU execution speed, or close to it.

From GPU-FPX to NixNan

GPU-FPX (HPDC 2023)

- Separate detector + analyzer (flow analysis)
- Exits early on first exception per thread
- No FP16 or Tensor Core support

NixNan (evolved from GPU-FPX)

- Single component: detects *and* flow-analyzes
- Finds *all* exceptions — never quits early
- Full FP16 and Tensor Core support

- Continuously refined against real workloads — this tutorial's own four case studies found and fixed two more bugs in NixNan itself (Slide 12).

Where NixNan Hooks Into the Compilation Pipeline

$$\text{CUDA C code} \xrightarrow{\texttt{nvcc}} \text{PTX code} \xrightarrow{\texttt{ptxas}} \text{SASS code}$$

- Small matrix multiplications $D = A \cdot B + C$ are performed **directly by Tensor Core hardware** — not expressed as ordinary scalar FP instructions at all.
- NixNan instruments **SASS** (the actual machine code), not the source or PTX: it sees exactly what runs on the GPU, *after* every compiler optimization.
- Consequence: **closed/precompiled libraries can be "X-rayed"** too — cuBLAS, cuSOLVER kernels included, no source needed.

How NixNan Instruments: NVBit + `LD_PRELOAD`

- NixNan is an **NVBit** (NVIDIA Binary Instrumentation Tool) tool.

```
LD_PRELOAD=nixnan.so ./your_precompiled_cuda_app
```

- NVBit intercepts every CUDA driver `Kernel Launch` call and gives NixNan a callback *before* that kernel's SASS runs.
- The SASS rewrite (inserting the checking calls) happens only the **first time** a given kernel function is seen — later launches of the same kernel reuse the already-instrumented code, so the rewrite cost is paid once, not per launch.
- Detects exceptions in **both SIMT cores and Tensor Cores**.

Detecting Exceptions: Bit-Pattern Classification

- For every FP-classified instruction, NixNan inserts a call to a device-side checker *before* and *after* it executes.
- Classification is pure bit-masking on the operand's exponent/mantissa — **no floating-point arithmetic**, so the check itself cannot introduce rounding artifacts:

```
uint32_t float_is_nan(uint32_t f) {
  return (float_exp(f)==0xFF && float_mant(f)!=0) ? E_NAN : 0;
}
```

- Per-lane exception bits are reduced across the warp (`__shfl_sync`); only one lane per warp records, avoiding 32-way redundant writes.
- First occurrence of a given (instruction, exception-type) pair is pushed live through a device→host channel to a receiver thread.

What NixNan Detects

Category	Meaning
NaN	Not-a-number
Infinity / −Infinity	Overflow (reported with sign)
Subnormal	Denormalized (near-zero) value
Division by zero	Divisor operand exactly zero

- Across **FP16, BF16, FP32, FP64** (FP8 is future work).
- Across **both SIMT-core scalar instructions** (FADD, FFMA, DFMA, ...) **and Tensor Core MMA instructions** (HMMA, confirmed empirically — see Case Study 3).
- Optional `INSTR_MEM=1`: also instrument global-memory **stores**, to confirm an exception was actually *written*, not just transiently computed.

Key Environment Variables

Variable	Effect
`LD_PRELOAD`	Points to `nixnan.so`; how the tool attaches at all
`INSTRUMENT_EXCEPTIONS`	Master switch for NaN/Inf/Subnormal/Div0 checking (default **on**)
`INSTR_MEM`	Also check global-memory stores
`HISTOGRAM` / `BIN_SPEC_FILE`	Exponent-range (binade) histogramming, Slide 10
`SAMPLING`	Instrument only every *N*th launch of a repeated kernel
`MAX_ERRORS`	Self-terminate after this many exception reports
`LOG_KERNELS`	**Disables** all instrumentation; just logs kernel launches + timing
`TOOL_VERBOSE`, `ENABLE_FUN_DETAIL`	Extra diagnostic printing at setup / per-launch
`FUNCTION_WHITELIST` / `_BLACKLIST`	Restrict instrumentation to (or exclude) named kernels

The Exponent-Histogram Mechanism

- `BIN_SPEC_FILE`: a JSON spec naming, per format, exponent ranges to watch, a report `count`, and (optionally) `doublings`.
- Every FP value updates a `[format][exponent]` histogram (powers the `"Exponent range for f16: [-9, 8]"` summary line).
- If the exponent falls in a watched bin, a per-bin counter increments; every `count`-th hit sends a **threshold report**.
- **A real blind spot**: valid bins explicitly exclude the zero and infinity exponent codes. **A value that jumps straight from finite to Inf in one instruction never lands in any bin** — invisible to this mechanism (see Slide 14).

Adaptive Doubling: an Exponentially Decaying Report Rate

- Reporting on *every* occurrence at `count:1` floods the log for a hot kernel. `doublings` fixes this:
  - Start reporting every `count` occurrences.
  - Each report **doubles** that bin's own threshold.
  - After `doublings` doublings, the *next* crossing **resets** every bin in that kernel back to `count`, and the cycle repeats.
- Example, `count:1, doublings:6`: report on the 1st, 2nd, 4th, 8th, 16th, 32nd, 64th occurrence, then reset.
- Used throughout Case Studies 1 and 3 to keep hot-kernel logs legible while still sampling every order of magnitude of occurrence.

Dogfooding NixNan: Two Concurrency Bugs Found & Fixed

While porting `doublings` for this tutorial, two real bugs were found **in NixNan itself**:

1. **Doubling-transition race**: `threshold` and `times_doubled` are shared, mutable state read-then-written by every GPU thread that crosses them — a classic check-then-act race. Evidence: threshold values observed jumping *backward* ($2 \to 1$ without ever reaching 4). **Fix**: gate both transitions with `atomicCAS` so exactly one thread performs each transition.
2. **Torn stdout**: the exception-report thread and the histogram-report thread both write multi-part « chains to the same stream with no synchronization — lines from two different reports interleave mid-message. **Fix**: assemble each report as one string, write it under a single mutex-protected `nnout_line()` call.

Lesson: exercising a tool's own adaptive/concurrent features hard enough surfaces real bugs in the tool, not just the target program.

`autonixnan`: A 3-Phase Automated Wrapper

- A Python wrapper around `nixnan.so` — launches the target program multiple times with different env-var combinations, stitches logs.
- **Phase 1 — baseline**: `LOG_KERNELS` mode; catalog every kernel and its wall-clock time, *zero* instrumentation overhead.
- **Phase 2 — extreme-value scan**: `HISTOGRAM=1`, exceptions off; find which kernels touch extreme exponents.
- **Phase 3 — deep dive**: full exception instrumentation, optionally whitelisted to the kernels Phase 2 flagged.
- Convenient for large, unfamiliar binaries (cuBLAS/cuSOLVER-heavy) — but see the next slide for its main failure mode.

A Real Blind Spot: Hard Overflow Jumps

- Case study: a mixed-precision Cholesky solver forced into overflow (`-noscale -bscale 1e5`) — confirmed by the *program's own* internal check (`nonfinite=256`) on the very first iteration.
- `autonixnan`'s Phase 2 scan **never flagged the responsible kernel** — it flagged an unrelated FP64 QR kernel used only for test-matrix generation. Phase 3's whitelisted deep-dive then reported **zero FP16 exceptions**.
- Why: the histogram mechanism's bins exclude zero/infinity by design (Slide 10) — a one-step jump to `Inf` leaves no gradually-climbing exponent trail for the scan to catch.
- **Fix**: run `nixnan.so` **directly**, unwhitelisted, no `autonixnan` — found the real exceptions immediately.
- *Guidance: prefer direct `nixnan.so` whenever the target is small enough to instrument in full.*

Real-World Impact: the SRU NaN Bug

- **SRU** (Simple Recurrent Unit): a CUDA implementation of a recurrent unit used from PyTorch.
- Open issue: `github.com/asappresearch/sru/issues/193` — users reported **NaN outputs** from the library's own example code.
- Case study performed by Dr. Xinyi Li (GPU-FPX's original author), later adapted to run under NixNan.
- This is not a synthetic teaching example — it is a **real, user-filed bug** in a published library.

SRU Case Study: Root Cause and Fix

- Running the reported-buggy example under NixNan: **NaN exceptions written into memory** appear almost immediately — far upstream of where the user first observed corrupted output.
- Following the exception back to source: an **uninitialized tensor**.

```
torch.randn(20, 32, 128).cuda() # was uninitialized memory before the fix
```

- Initializing it (instead of leaving it as garbage device memory) eliminates the NaNs. Before/after NixNan logs confirm: exceptions written into memory before the fix, none after.
- *Exactly the pattern this tutorial is about: an exception generated far from where a user would ever look, made visible by instruction-level instrumentation.*

Four Case Studies

Case study	Workload	Focus
`rd_nixnan`	Reaction-diffusion PDE solver	FP64 vs. FP32 dynamic range
`lu_solve_gpu`	LU factorization, partial pivot	Ill-conditioning $\rightarrow$ NaN
`tc_matmult`	Tensor-Core GEMM (WMMA)	Tensor cores + real overhead
`mpir_gpu`	Mixed-precision iterative refinement	Precision hierarchy at scale

All four run on an NVIDIA RTX 3090 / RTX-class Ampere GPU (sm_80/sm_86), instrumented with `LD_PRELOAD=nixnan.so`.

Case Study 1: `rd_nixnan` — Reaction-Diffusion

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2} + \lambda u, \qquad \lambda = 25,\ T = 4$$

- Growth factor by t=4: $e^{100} \approx 2.7 \times 10^{43}$ — inside FP64's range, well past FP32's ($\approx 3.4 \times 10^{38}$).

	FP64	FP32
Max $\lvert u \rvert$ at $t = 3.52$	1.11×10^{38}	1.10×10^{38}
Max $\lvert u \rvert$ at $t = 3.60$	8.09×10^{38}	Overflows to Inf
Max $\lvert u \rvert$ at $t = 4.00$	1.70×10^{43}	NaN (propagated)
NixNan exceptions	0	36 NaN / 16 Inf / 40 −Inf

FP64 completes cleanly; FP32 overflows between $t = 3.52$ and $t = 3.60$, then NaN propagates for the rest of the run — the CPU tutorial's lesson, reproduced on GPU.

`rd_nixnan`: A Silent-Failure Wrinkle

- The reduction that reports the running max mirrors `std::max_element`'s NaN hazard on purpose: `v > best` (not `fmax`).
- Once the array goes non-finite, *every* comparison against `best` fails — `best` never leaves its seed value, `0.0` (a boundary point, pinned by the Dirichlet condition).
- Result: the program prints a **plausible, finite** `0.0` from t=3.60 onward — not `nan`, not `inf`.
- This **silently defeats the program's own** `std::isfinite()` **self-check** — it never fires, because the reported value genuinely is finite.
- NixNan is unaffected: it instruments the FADD/FFMA *inside* the step kernel, before the corrupted value ever reaches the reduction. It reports the real NaN/Inf (and, with `INSTR_MEM`, confirms 36,928 NaN values actually written to global memory) regardless.

Case Study 2: `lu_solve_gpu` — LU Factorization

$PA = LU$, partial pivoting, forward/backward substitution, 20×20 system, "snap near-epsilon values to zero" elimination rule.

	`matrix.csv` (well-cond.)	`bad_matrix.csv` (ill-cond.)
Residual $\|Ax - b\|_2$	9.68×10^{-16}	`nan`
FP64 NaN (repeats)	0	316 (2171)
FP64 Infinity	0	4
FP64 Division-by-zero	0	4

- Mechanism: a near-zero pivot is snapped to exactly `0.0` by the epsilon-clamp rule; the subsequent division by that zero pivot produces $\pm\infty$, and NaN propagates through the rest of the solve.

`lu_solve_gpu`: A Binary-Instrumentation False Positive

- The ill-conditioned run also reports **6 FP32 NaN detections** — despite the solver computing exclusively in `double`.
- Root cause, found via the flagged SASS: `FFMA R8, RZ, R5, R3` — `ptxas`'s "multiply-by-the-zero-register" idiom for a **register move** ($0 \times R5 + R3 = R3$), used to shuffle the two 32-bit halves of a 64-bit double across the register file.
- NixNan correctly classifies `FFMA` as an `f32` instruction and correctly notices `R5`'s bit pattern decodes as NaN — but that 32-bit word is *half of a double*, never interpreted as an `f32` value by the program.
- **Takeaway**: a nonzero count in a format the source never uses does not necessarily mean that format's arithmetic ran — a known class of false positive for any SASS-level FP exception detector.

Case Study 3: `tc_matmult` — Tensor Cores Under NixNan

- WMMA-based FP16/BF16 GEMM (row-major × row-major), plus hand-written SIMT FP32/FP16 comparison kernels.
- **Confirmed empirically**: NixNan's instruction table matches tensor-core `HMMA.16816.F32` directly — tensor-core MMA instructions are *not* invisible to this instrumentation, despite operating on packed fragment registers rather than scalar values.

Kernel	Latency	vs. SIMT (same precision)
FP16, tensor core	≈ 0.178 ms	
FP16, SIMT	≈ 0.865 ms	$\approx 4.9\times$ faster (TC)
BF16, tensor core	≈ 0.177 ms	
FP32, SIMT	≈ 0.845 ms	$\approx 4.8\times$ faster (TC)

(All 1024^3, uninstrumented baselines.)

`tc_matmult`: Measuring NixNan's Real Overhead

Four instrumentation levels, one FP16 tensor-core kernel (128^3, row-major $\times$ row-major):

Instrumentation level	Latency	Slowdown
No NixNan (baseline)	0.008 ms	$1\times$
Sampled + histogram (misleading, see next slide)	0.015 ms	$\approx 1.9\times$
Unsampled, default exception checking only	0.706 ms	$\approx 88\times$
Unsampled, exceptions + `count:1` histogram	257.009 ms	$\approx 32{,}126\times$

- Finest-granularity histogramming (`count:1`) is **drastically** more expensive than exception checking alone — every single occurrence in a watched bin sends a report.
- *Instrumentation cost is a real design parameter, not an afterthought — choose granularity deliberately.*

tc_matmult: SAMPLING and a Downsizing Lesson

- The "sampled" row above (`SAMPLING=128`) looks almost free (1.9×) — but this is misleading: this kernel launches only ∼88 times total, so `SAMPLING=128` instruments only launch #0, never one of the 80 *timed* repeats.
- **Original attempt** at 1024^3 with `count:1`: the kernel executes ∼262,000 HMMA instructions per launch — the resulting report volume through NixNan's channel meant the run never finished in 10+ minutes.
- **Fix**: rerun at 128^3 (1/512 the work) — tractable in under a minute, same code, same watched exponent ranges (re-verified empirically for the smaller sample count).
- At 128^3 the tensor-core-vs-SIMT speedup itself collapses to ≈1.4× (fixed launch overhead dominates at this size) — a reminder that microbenchmark *size* shapes what you can conclude.

Case Study 4: `mpir_gpu` — Mixed-Precision Iterative Refinement

- Carson–Higham iterative refinement Cholesky solve, n=4096, six formats: F64, F32, TF32, F16, BF16-truncated (B16T), BF16-stochastic (B16S).
- Ill-conditioned regime (κ up to 10^5): **all** formats break down (`niter=-1`) — expected, by design of the test matrices.
- Well-conditioned regime (n=256): convergence-iteration count tracks precision directly.

κ	F64	F32	TF32	F16	B16T	B16S
10	0	0	0	2	2	4
10^3	1	1	1	3	10	12

Iterations to reach relative error $< 10^{-6}$ — lower precision needs more refinement steps, especially as κ grows.

mpir_gpu: Subnormal Storms and Extreme Overhead

Overflow stress test ($\kappa = 10^3$, amplified residual $b_{\text{scaled}} = 10^5 b$), NixNan-instrumented:

Format	Baseline	Instrumented	Slowdown
F16	90 ms	76,238 ms	847×
B16T	155 ms	79,530 ms	513×
F32	1.5 ms	82,727 ms	55,152×

- F16/B16T overhead traces to ∼128 **subnormal (underflow)** exceptions per GEMM inside HMMA tensor-core kernels — F16's 5-bit exponent (min $\approx 6\times10^{-5}$) is simply too narrow once intermediate values are scaled.
- F32's overhead is driven by NaN/Inf in QR preprocessing (Householder reflections), *not* tensor cores — a different failure mode entirely, same tool catches both.

`mpir_gpu`: Exceptions Are Not the Same as Failure

- At $\kappa = 10^3$, **every** format shows NixNan-detected exceptions (subnormal, div0, NaN, infinity) in QR/factorization — **yet all still converge**, given enough refinement iterations.
- Sweeping κ upward: B16T/B16S **fail** (breakdown, `niter=-1`) at $\kappa \geq 2000$; F32/F64 keep converging through $\kappa = 5000$ (limit of the tested range).
- **Key insight**: the presence of exceptions is not the convergence-killer. It is the format's **precision capacity** — whether iterative refinement has enough headroom to correct the damage —that determines success or failure.
- *NixNan tells you exceptions are happening; it does not by itself tell you whether that matters for your algorithm's correctness. That judgment still belongs to the practitioner.*

Cross-Cutting Lessons for HPC Practitioners

- **Overflow-then-NaN-propagation** is the single most common failure mode across all four case studies — and it is exactly the pattern instruction-level instrumentation is built to catch early.
- **A program's own diagnostics can be fooled**: NaN-hazard reductions (`std::max_element`-style comparisons) can mask real non-finite data behind a plausible finite summary value (`rd_nixnan`).
- **Instrumentation overhead varies by orders of magnitude**: from $\approx 2\times$ (sparse sampling) to $\approx 30{,}000\times$ (unsampled, finest histogram granularity) to $\approx 55{,}000\times$ (exception-dense kernels) — always measure it for your own workload.
- **Binary-level detection has real false positives** (`lu_solve_gpu`'s register-move idiom) **and real blind spots** (the histogram scan's zero/Inf exclusion) — know both before trusting a report uncritically.

Practical Guidance

- **Start here for a small/medium target**: run `nixnan.so` **directly**, unwhitelisted, default `INSTRUMENT_EXCEPTIONS=1` — no scan-phase blind spot.

```
LD_PRELOAD=./nixnan.so ./target [args...]
```

- **For a large/unfamiliar library-heavy binary**: use `autonixnan`'s 3-phase workflow to triage first, but verify its Phase-2 kernel selection against a direct run before trusting a "clean" Phase-3 result.
- **For a hot kernel**: use `BIN_SPEC_FILE` with `doublings` rather than `count:1` unsampled — keeps logs legible and overhead bounded while still sampling every decade of occurrence.
- **Always cross-check** a per-format exception count against what the source code actually computes in that format.

Conclusion

- NixNan brings instruction-level, no-hardware-trap-required FP exception detection to both SIMT cores *and* Tensor Cores on NVIDIA GPUs — including inside closed, precompiled libraries.
- Four independent case studies — a PDE solver, an LU factorization, a tensor-core GEMM, and a mixed-precision iterative refinement solver — all reproduce the same central lesson: **low precision and ill-conditioning fail the same way, overflow then NaN propagation, and that failure is often invisible until instrumented.**
- A real, user-filed library bug (SRU) was found and fixed the same way.
- NixNan itself is under active development — this tutorial's own case studies found and fixed two concurrency bugs in the tool.

`https://github.com/parfloat/nixnan`